PrintStop v2.0 프로젝트 심층 검토 및 발전 방향

핵심 질문: PrintStop은 이미 존재하는 3D 프린팅 실패 감지 AI보다 무엇을 더 잘할 수 있는가?

현재 PrintStop의 기본 아이디어는 유지하겠습니다.

- 문제: FDM 3D 프린팅 중 서포트 파손, 적층 실패, 출력물 이탈 등으로 발생하는 시간·필라멘트 손실
- 기술: 카메라 영상 + PatchCore 기반 딥러닝 이상탐지 + G-code 출력 단계 정보
- 목표: 실패를 조기에 발견하고, 정상 출력과 실제 이상을 구분해 사용자에게 경고
- 구현 범위: 초기 버전은 이상 감지·알림·기록, 자동 정지는 안전성 검증 이후 확장

먼저 결론부터 말하면, PrintStop은 충분히 도전해 볼 만한 프로젝트이지만, 현재 정의만으로는 경쟁력이 확실하다고 보기 어렵습니다.

이미 AI로 3D 프린팅 실패를 감지하는 제품들이 있기 때문입니다. 차별점을 단순한 이상 감지에서 출력 공정 정보를 활용한 조기 실패 판단으로 좁히고, 그것이 실제로 효과적인지를 검증하는 것이 중요합니다.

1. 경쟁 서비스는 이미 무엇을 해결하고 있을까?

Obico

카메라 기반으로 스파게티 현상, 출력물 이탈, 레이어 밀림, 노즐 필라멘트 뭉침 등을 감지합니다. 알림과 자동 일시정지도 지원합니다.



Obico

OctoEverywhere Gadget

카메라 영상으로 다양한 출력 실패를 감지하고 알림을 제공합니다. 일부 고급 기능에는 자동 일시정지와 레이어·접착 문제 감지가 포함됩니다.



octoeverywhere.com

Bambu Lab X1-Carbon

프린터 자체에 카메라 기반 출력 실패 감지 기능을 제공합니다. 다만 특정 기기와 감지

가능한 실패 유형에 제약이 있습니다.



Obico

여기서 중요한 발견이 있습니다.

비교 질문	현재 판단
AI로 출력 실패를 감지한다	이미 상용화됨
카메라만으로 감지한다	차별성이 약함
사용자에게 경고한다	이미 상용화됨
출력 실패 시 자동 정지한다	이미 구현된 제품 존재
서포트의 구조적 이상을 조기에 감지한다	특화 성능 검증 필요
G-code의 예상 출력 상태와 실제 영상을 결합한다	차별화 가능성이 있으나 우위 미검증
기존 제품보다 더 빠르고 정확하게 감지한다	비교 실험 없이는 주장 불가

특히 G-code를 활용한다는 사실 자체가 세계 최초의 기술이라는 의미는 아닙니다. 우리가 증명해야 하는 것은 G-code를 추가했을 때 카메라만 사용하는 방식보다 얼마나 좋아지는가입니다.

2. 프로젝트를 검증하기 위해 반드시 던져야 할 12가지 질문

현재까지 확인된 정보에 따른 판단입니다. '미검증'은 기술적으로 불가능하다는 뜻이 아니라, 실제 출력 실험으로 증명해야 한다는 의미입니다.

A. 고객과 문제의 본질

Q1. 누가 이 제품을 가장 절실하게 필요로 할까?

답변: 장시간 출력하거나 여러 프린터를 운영하며, 서포트 실패 때문에 반복 출력 비용이 발생하는 사용자가 가장 유력합니다.

일반 취미 사용자보다 대학 연구실, 메이커스페이스, 시제품 제작팀, 소규모 출력 대행업체가 초기 고객 후보로 적합합니다.

다만 이들이 실제로 PrintStop을 구매하거나 설치할지는 아직 검증되지 않았습니다.

Q2. 사용자가 겪는 가장 큰 고통은 정확히 무엇인가?

실패 자체보다 실패한 상태를 오랫동안 모르고 계속 출력하는 것이 PrintStop이 직접 해결할 수 있는 문제입니다.

예를 들어 서포트가 3시간째 파손됐는데 8시간짜리 출력이 계속된다면, 이후의 시간과 재료를 추가로 낭비할 수 있습니다.

하지만 조기에 감지해도 이미 파손된 출력물이 복구되는 것은 아닙니다. 따라서 '실패 방지'가 아니라 '실패 이후 낭비 최소화'가 1차 가치입니다.

Q3. 고객이 이미 Obico를 사용하고 있다면 왜 바꿔야 할까?

현재는 설득할 근거가 부족합니다.

PrintStop이 특정 서포트 파손 사례를 더 일찍 감지하거나, 정상적인 형상 변화에 대한 오경보를 줄였다는 실험적 증거가 필요합니다.

따라서 고객 설득의 핵심은 AI 모델 종류가 아니라 추가로 절약할 수 있는 출력 시간과 실제 오경보 감소량이어야 합니다.

Q4. 사용자가 반드시 모든 종류의 실패를 감지하길 원할까?

반드시 그렇지는 않습니다.

작은 적층 결함, 표면 품질 저하, 첫 층 접착 불량, 서포트 파손은 원인과 감지 방법이 다릅니다. 처음부터 모두 해결하려 하면 데이터 구축과 검증이 어려워집니다.

1차 핵심 대상으로 '출력 중 서포트의 파손·기울어짐·이탈'을 설정하고, 다른 실패는 확장 기능으로 관리하는 편이 좋겠습니다.

B. 기술적으로 구현할 수 있는가?

Q5. PatchCore만으로 서포트 파손을 감지할 수 있을까?

가능성은 있지만, 가장 큰 기술적 불확실성이 있습니다.

PatchCore는 정상 이미지의 특징을 저장한 뒤 새로운 이미지가 얼마나 다른지 판단합니다. 일반적인 산업 이상탐지에서는 유용한 방식입니다.



Anomalib documentation

문제는 3D 출력물이 매 순간 성장한다는 점입니다.

정상 출력

정상적으로 높이가 증가

서포트 이상

기울어지거나 붕괴

개념도: 정상적인 높이 증가와 구조 이상을 구분해야 합니다.

출력 10층의 정상 모습과 100층의 정상 모습도 다릅니다. 학습할 때 이 변화를 고려하지 않으면 정상 출력에 이상 점수를 줄 수 있습니다.

Q6. G-code를 결합하면 이 문제를 해결할 수 있을까?

일부 개선할 수 있지만, G-code만으로 해결되지는 않습니다.

G-code에서 현재 레이어와 서포트가 출력되는 구간을 추정해 딥러닝 결과를 해석하는 데 사용할 수 있습니다.

다만 실제 출력 시간과 G-code상의 진행 위치가 정확히 일치해야 하고, 카메라 영상 좌표와 출력 좌표의 정합도 필요합니다.

PrusaSlicer는 레이어·압출 유형 등의 정보를 G-code 주석에 제공하지만, 정보의 종류와 정밀도는 슬라이서마다 다릅니다.



Prusa Knowledge Base

따라서 처음부터 G-code로 완전한 예상 이미지를 렌더링하는 대신, 현재 레이어 + 서포트 영역 + 이상 발생 지속시간을 결합하는 방식을 권합니다.

Q7. 서포트 파손을 실제로 조기에 알아차릴 수 있을까?

서포트가 보이는 경우에 한해 가능성이 있습니다.

정상일 때의 서포트 위치에서 구조가 사라지거나 기울어지는 모습은 시각적으로 확인할 수 있습니다. 하지만 출력물에 가려지거나, 내부에서 파손되거나, 아직 외형 변화가 없는 경우에는 일반 카메라만으로 판단하기 어렵습니다.

따라서 프로젝트 초기 조건으로 고정 카메라, 일정한 조명, 서포트가 노출된 출력물을 설정해야 합니다.

Q8. 실시간 감지와 프린터 제어는 가능한가?

기술적으로 가능합니다. 단, 프린터와 연결 방식에 따라 달라집니다.

OctoPrint는 출력 상태 조회, 일시정지, 취소 API를 제공하며, Klipper 계열의 Moonraker 역시 출력 제어 API를 제공합니다.

OctoPrint main documentation

+1

그러나 감지 결과가 잘못됐을 때 정상 출력까지 중단되는 위험이 있습니다. 먼저 경고만 제공하는 방식으로 신뢰성을 검증하는 것이 합리적입니다.

C. 제품으로서 실제 효과가 있는가?

Q9. 탐지 정확도가 95%면 충분할까?

아닙니다. 무엇을 기준으로 측정한 95%인지가 더 중요합니다.

실패 프레임이 많이 포함된 데이터에서 높은 정확도를 달성해도 실제 출력 중에는 오경보가 빈번할 수 있습니다.

따라서 프레임 정확도보다 실패 이벤트 검출률, 정상 출력 시간당 오경보 수, 실제 파손 이후 경보까지 걸린 시간을 평가해야 합니다.

Q10. 다른 제품보다 빨리 감지하면 반드시 가치가 있을까?

반드시 그렇지는 않습니다.

예를 들어 기존 서비스가 실패 2분 후 경고하고 PrintStop이 1분 후 경고한다면, 기술적으로는 개선이지만 고객의 실제 편익은 작을 수 있습니다.

반대로 기존 서비스가 서포트 파손을 놓치고 스파게티 현상이 나타난 뒤에야 경고하는 사례를 PrintStop이 먼저 발견한다면 가치가 커집니다.

우리가 찾아야 하는 것은 탐지 속도가 빠른 사례가 아니라, 기존 방식이 놓치거나 늦게 발견하는 의미 있는 실패 유형입니다.

Q11. 카메라와 GPU가 추가되면 오히려 비용이 증가하지 않을까?

맞습니다. 기존 PC와 USB 카메라만 활용할 수 있으면 부담이 작지만, 전용 장치나 고가의 GPU가 필요하면 설치 비용이 커집니다.

초기 버전은 1대의 카메라와 기존 PC에서 동작하도록 설계하고, 저사양 추론 최적화는 후속 단계로 두겠습니다.

Q12. 이 프로젝트가 실패했다는 판단은 언제 내릴까?

다음 중 하나가 반복적으로 확인되면 현재 접근을 재검토해야 합니다.

- 정상적인 형상 성장과 파손을 안정적으로 구분하지 못하는 경우
- 서포트 파손이 카메라에 보이지 않아 감지 자체가 어려운 경우
- G-code 결합 전후 성능 차이가 실질적으로 없는 경우
- 기존 솔루션 대비 감지 시점이나 오경보 측면의 이점이 없는 경우
- 감지에 성공하더라도 고객이 실제로 절약하는 자원이 미미한 경우

이 기준이 있어야 단순히 모델만 계속 바꾸는 상황을 피할 수 있습니다.

3. 그렇다면 무엇을 추가해야 할까?

제가 제안하는 발전 방향은 기능을 많이 추가하는 것이 아니라, 서포트 이상을 조기에 발견하는 데 꼭 필요한 기능을 강화하는 것입니다.

제안 1. 단순 이상탐지에서 '공정 단계 인식형 이상탐지'로 발전

PrintStop v2.1 제안 아키텍처

실시간 카메라 영상

출력 상태 관찰

PatchCore

이상 점수 및 히트맵

G-code 해석

레이어·서포트 구간

공정 정보를 반영한 판단

서포트 관심영역 + 출력 단계 + 지속시간 검증

정상

관찰 유지

의심

재검증

이상

경고·기록

예를 들어 동일한 이미지 이상 점수가 발생해도, 그 위치가 현재 출력 단계에서 정상적으로 형상이 성장하는 위치인지, 이미 형성이 끝난 서포트의 위치인지에 따라 다르게 판단할 수 있습니다.

단, 초기 구현에서는 서포트 관심영역을 수동 지정하고 G-code로 검사 시점을 정하는 것이 현실적입니다. 이후 G-code의 서포트 압출 경로를 이용해 관심영역을 자동 생성하는 방식으로 발전시킬 수 있습니다.

제안 2. '서포트 파손 → 출력 실패'의 시간 순서를 탐지

이 프로젝트에서 특히 연구 가치가 있는 부분이라고 생각합니다.

출력 실패 진행 단계

1

정상

2

서포트 이상

3

적층 불량

4

출력 실패

PrintStop의 목표: 2단계에서 경고하여 3~4단계까지 진행하며 발생하는 추가 낭비를 줄이는 것

모든 출력 실패가 이 순서로 진행하는 것은 아닙니다. 서포트 이상이 먼저 나타나는 사례에 한정한 검증 가설입니다.

핵심은 스파게티 현상을 감지하는 것보다 앞선 이상을 포착할 수 있느냐입니다.

실제 서포트 변형이 처음으로 관찰된 시점과 시스템이 경고한 시점, 이후 명확한 출력 실패가 발생한 시점을 각각 기록해야 합니다.

이를 통해 PrintStop이 얼마나 일찍 위험을 발견했는지 정량적으로 설명할 수 있습니다.

제안 3. 결과만 보여주는 AI가 아니라 판단 근거를 제공하는 AI

단순히 '실패 확률 87%'라고 표시하는 것보다 다음 정보를 보여주는 편이 활용 가치가 높습니다.

PrintStop — 이상 감지 이벤트

검토 필요

현재 레이어

82 / 250

이상 지속시간

18s

서포트 관심영역에서 이상 검출

해당 레이어의 출력 공정 정보 확인

연속 프레임에서 이상 지속 여부 확인

권장 조치: 사용자 영상 확인

실제 측정값이 아닌 화면 구성 예시입니다.

이러한 이벤트 기록은 사용자에게 유용할 뿐 아니라, 모델이 왜 오탐했는지 분석하는 자료로도 사용할 수 있습니다.

4. 구현해야 하는 기능의 우선순위

기능	우선순위	판단
실시간 카메라 수집	필수	구현 난이도 낮음
PatchCore 이상탐지	필수	구현 가능, 성능 검증 필요
이상 위치 히트맵	필수	PatchCore 활용 가능
G-code 레이어 파싱	필수	구현 가능
서포트 관심영역 지정	필수	초기에는 수동 방식
이상 지속시간 기반 경고	필수	오경보 완화 목적
이벤트 이미지·시간 기록	필수	검증에 반드시 필요

OpenCV·PatchCore·융합 모델 비교	필수	기술적 가치 입증
서포트 영역 자동 추출	추가	슬라이서별 처리 필요
G-code 예상 형상 렌더링	후속	좌표 정합이 어려움
프린터 자동 정지	후속	안전성 검증 필요
여러 프린터 동시 관리	후속	MVP에서 제외

여기서 주의할 점은 모델 구현보다 카메라 좌표와 실제 G-code 진행 상태를 맞추는 일이 더 오래 걸릴 수 있다는 것입니다.

OctoPrint에는 레이어 정보를 추적하는 DisplayLayerProgress 플러그인이 있으며, 이를 활용하거나 구현 방식을 참고할 수 있습니다.



plugins.octoprint.org

따라서 첫 버전에서는 하나의 프린터, 하나의 슬라이서, 고정 카메라를 기준으로 시작하길 권합니다.

5. 가장 중요한 실험: G-code를 결합할 가치가 있는가?

이 프로젝트의 기술적인 설득력은 단순히 딥러닝을 사용했다는 사실에서 나오지 않습니다.

G-code라는 공정 정보가 추가되었을 때 실제 검출 성능이 개선되는지를 증명하는 것이 가장 중요합니다.

실험	적용 방식	검증 목적
A	OpenCV 기반 영상 변화 탐지	기본 영상처리의 한계 확인
B	PatchCore 단독	딥러닝의 효과 확인
C	PatchCore + 레이어 정보	정상 형상 성장 오탐 감소 여부
D	PatchCore + 레이어 + 서포트 ROI + 시간 검증	최종 제안 시스템 효과

가능하다면 Obico 또는 Gadget을 동일한 출력 조건에서 함께 실행해 참고 기준으로 추가하는 것도 좋겠습니다. 특히 Gadget은 외부 프로젝트에서 사용할 수 있는 실패 감지 API를 제공합니다.



OctoEverywhere API Docs

단, 외부 서비스의 모델 입력 주기와 동작 환경이 다를 수 있으므로 비교 조건을 명시해야 합니다.

성공 여부를 판단할 정량적 기준

아래 수치는 기존 연구의 검증된 성능이 아니라, 첫 번째 실험을 설계하기 위한 잠정 목표입니다.

서포트 실패 이벤트 검출률	80% 이상 목표
정상 출력 1시간당 오경보	0.05회 이하 목표
감지 지연시간	60초 이내 목표
단독 PatchCore 대비 개선	오경보 또는 조기 감지 성능에서 유의미한 개선

스파게티 발생 이전 경고 비율 별도 측정

정상 출력은 최소 수십 시간, 실패 사례는 유형별 반복 실험으로 수집하는 것을 초기 목표로 잡을 수 있습니다. 다만 적은 실패 사례에서 80% 검출률을 달성했다고 일반적인 성능이 검증되는 것은 아닙니다.

또한 학습·평가 데이터를 단순히 같은 출력 영상의 프레임 단위로 무작위 분리하면 안 됩니다. 서로 다른 출력 작업과 조건을 기준으로 분리해야 성능이 과대평가되는 것을 줄일 수 있습니다.

장비를 손상시킬 수 있는 강제 파손 실험은 피하고, 과거 실패 영상과 안전하게 재현 가능한 사례부터 활용해야 합니다.

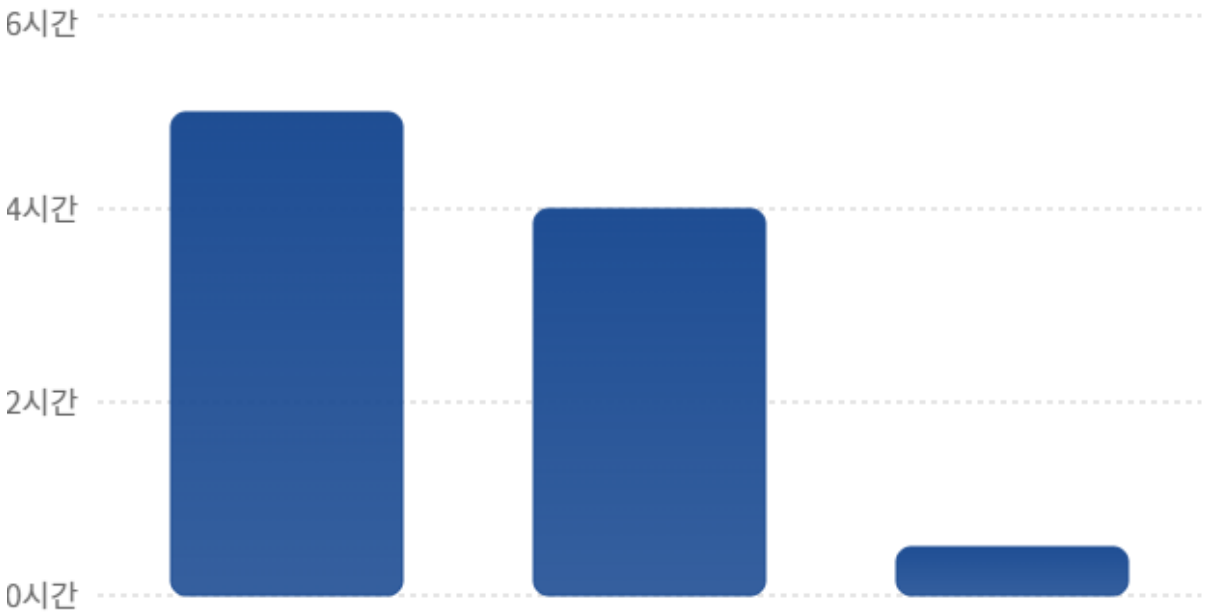
6. 고객의 페인을 정말 해결했는지 확인하는 방법

기술 검증만으로는 부족합니다. 고객에게 실제로 가치가 있는지도 확인해야 합니다.

예를 들어 8시간짜리 출력에서 3시간째 실패가 발생했다고 가정하겠습니다.

실패 발견 시점에 따른 추가 낭비 시간

8시간 출력, 3시간째 실패 가정 · 예시



PrintStop이 3시간 30분에 경고해 사용자가 즉시 중단했다면, 종료 후 발견하는 경우보다 4.5시간의 추가 출력을 줄일 수 있습니다.

그렇지만 이미 사용 중인 서비스도 같은 시점에 경고한다면 PrintStop만의 추가 가치는 없습니다.

결국 다음 두 가지를 구분해야 합니다.

기술적 성능: PrintStop은 이상을 얼마나 정확하고 빠르게 찾아내는가?

고객 가치: 기존 해결책보다 실제로 얼마나 더 많은 시간, 필라멘트, 관리 부담을 줄여 주는가?

이 질문은 예상 절감량이 아니라 실제 실험 기록과 사용자 인터뷰로 확인해야 합니다.

고객 검증에 필요한 질문

실제 프린터를 사용하는 사람들에게 다음을 물어보는 것을 권합니다.

- 최근 출력 실패는 어떤 유형이었으며 몇 시간 뒤에 발견했는가?
- 이미 프린터 내장 AI나 Obico와 같은 감지 서비스를 이용하고 있는가?
- 가장 큰 비용은 필라멘트, 출력 시간, 프린터 점유, 사람이 확인하는 시간 중 무엇인가?
- 잘못된 경고가 발생해도 조기 실패 감지를 위해 이 시스템을 사용하겠는가?
- 설치에 카메라와 별도 프로그램이 필요하다면 사용할 의향이 있는가?

여기서 중요한 것은 단순히 '좋은 아이디어네요'라는 답이 아니라, 기존에 겪은 구체적인 실패와 비용을 확보하는 것입니다.

7. 개발 범위는 이렇게 조정하는 것이 좋겠습니다

1단계 — 딥러닝 탐지 성립 여부 확인

고정 카메라와 동일 출력 조건에서 정상/이상 영상을 확보하고 PatchCore의 기본 탐지 성능을 확인합니다. 정상 형상 변화 때문에 과도한 이상 점수가 발생하면 모델 학습·검사 영역을 먼저 조정합니다.

2단계 — G-code 기반 조건부 탐지

레이어 정보와 서포트 관심영역을 결합합니다. 이 단계에서 단독 PatchCore보다 오탐이나 탐지 지연이 개선되는지 검증합니다.

3단계 — 모니터링 시스템 완성

현재 출력 상태, 이상 위치, 경고 기록, 사용자 확인 기능을 통합하고 실제 프린터 영상에서 동작시킵니다.

4단계 — 기존 방식과 비교 실험

오경보 수와 탐지 지연, 서포트 이상이 먼저 발견되는 사례, 실제 절약 가능 시간을 정리합니다. 이 결과로 프로젝트의 핵심 가설을 채택할지 판단합니다.

8. 최종 제안: PrintStop의 목표 문장을 수정하자

현재의 목표가 다음과 같다면,

카메라와 딥러닝을 활용하여 3D 프린팅 실패를 실시간으로 탐지하고 사용자에게 알리는 시스템

이 목표 자체는 나쁘지 않지만, 이미 존재하는 시스템과 크게 다르지 않습니다.

다음과 같이 바꾸는 것을 제안합니다.

PrintStop v2.1

G-code 공정 정보를 결합한 FDM 3D 프린팅 서포트 이상 조기 탐지 시스템

카메라 기반 딥러닝 이상탐지와 G-code의 레이어·서포트 출력 정보를 결합하여, 정상적인 출력 형상 변화와 구조적 이상을 구별하고 서포트 파손으로 인한 후속 출력 실패를 조기에 발견하는 시스템을 개발한다.

나아가 영상 단독 이상탐지 방식과 비교하여 오경보 감소, 탐지 시점 개선 및 잠재적 출

력 자원 절감 효과를 정량적으로 검증한다.

최종 평가

평가 관점	현 단계 판단
고객 및 문제의 명확성	비교적 높음, 실제 인터뷰 필요
기술 구현 가능성	조건을 제한하면 높음
딥러닝 활용의 적절성	높음, 다만 모델 선정 검증 필요
기술적 차별성	잠재력 있음, 검증 전
경쟁 제품 대비 우위	아직 불확실
프로젝트 완성 가능성	MVP 범위를 유지하면 양호
연구·포트폴리오 가치	비교 실험이 충실하면 높음

제 생각에는 PrintStop의 가장 큰 위험은 구현을 못 하는 것이 아니라, 구현에 성공해도 기존 제품보다 나은 이유를 설명하지 못하는 것입니다.

따라서 다음 작업에서는 PDF 디자인을 더 다듬기보다, 먼저 서포트 파손을 실제로 촬영할 수 있는 조건을 정하고 PatchCore 단독과 G-code 결합 방식의 차이를 확인하는 소규모 검증 실험을 설계하는 것이 좋겠습니다.

그 실험에서 개선 효과가 확인된다면 PrintStop은 단순 AI 시연이 아니라, 명확한 문제와 해결 근거를 갖춘 프로젝트로 발전할 수 있습니다.